EXPERIMENT 6B – TIME SERIES DATA ANALYSIS AND FORECAST

AIM

To analyse the monthly atmospheric CO_2 time series (Mauna Loa), study its trend, seasonality and stationarity, and forecast future values using Holt-Winters and SARIMA models.

ALGORITHM

1. Load the CO_2 dataset, convert weekly readings to a monthly series and fill missing values.
2. Plot the series with rolling mean and rolling standard deviation.
3. Decompose the series into trend, seasonal and residual components.
4. Test stationarity with the Augmented Dickey-Fuller (ADF) test; apply differencing and plot ACF/PACF.
5. Split the data into train and test (last 36 months for testing).
6. Fit Seasonal Naive (baseline), Holt-Winters and SARIMA(1,1,1)(1,1,1,12) models.
7. Compare models using MAE, RMSE and MAPE, and plot forecasts against actual values.
8. Refit the best model on the full data and forecast the next 24 months with a confidence interval.

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt, warnings
import statsmodels.api as sm
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error
warnings.filterwarnings("ignore")

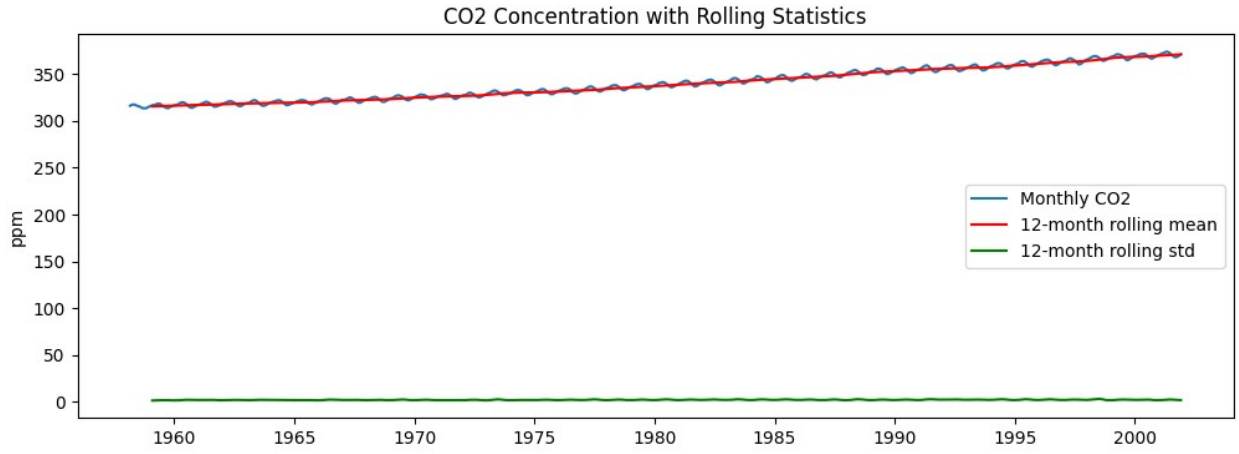
# Load dataset (built into statsmodels — no upload needed)
raw = sm.datasets.co2.load_pandas().data
print("Raw shape:", raw.shape, "| Missing values:",
raw.co2.isnull().sum())
ts = raw.co2.resample("MS").mean().interpolate()      # weekly ->
monthly, fill gaps
ts.name = "CO2 (ppm)"
print(ts.head(), "\n", ts.tail(), "\nMonthly shape:", ts.shape)
print(ts.describe())
```

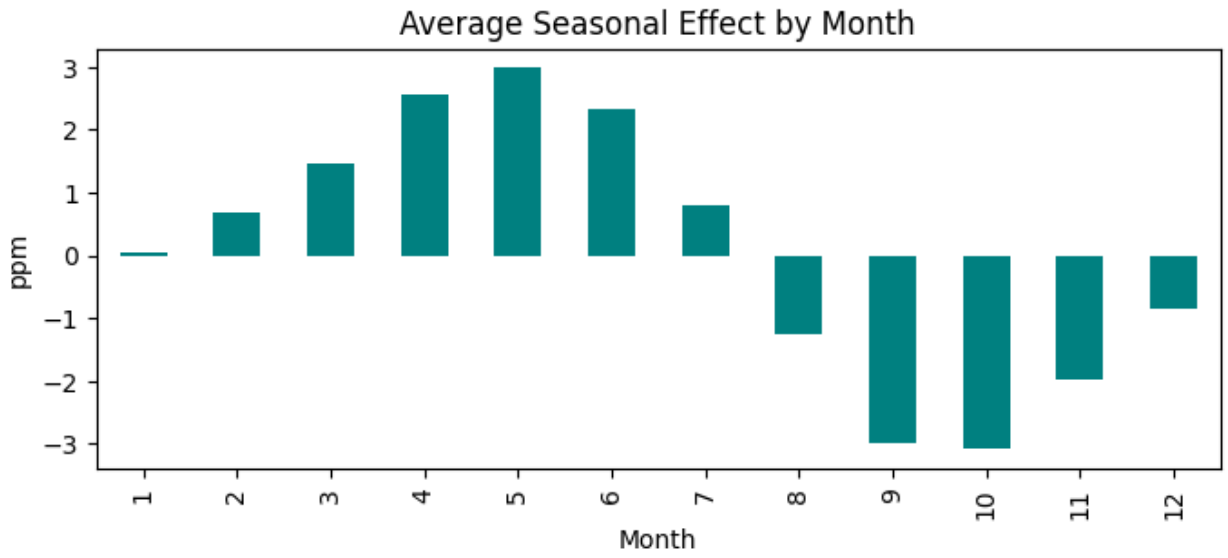
```
Raw shape: (2284, 1) | Missing values: 59
1958-03-01    316.100000
1958-04-01    317.200000
1958-05-01    317.433333
1958-06-01    316.529167
1958-07-01    315.625000
```

```
Freq: MS, Name: CO2 (ppm), dtype: float64
 2001-08-01    369.425
2001-09-01    367.880
2001-10-01    368.050
2001-11-01    369.375
2001-12-01    371.020
Freq: MS, Name: CO2 (ppm), dtype: float64
Monthly shape: (526,)
count    526.000000
mean     339.619202
std       17.116453
min      313.400000
25%      324.025000
50%      337.912500
75%      354.537500
max      373.800000
Name: CO2 (ppm), dtype: float64
```

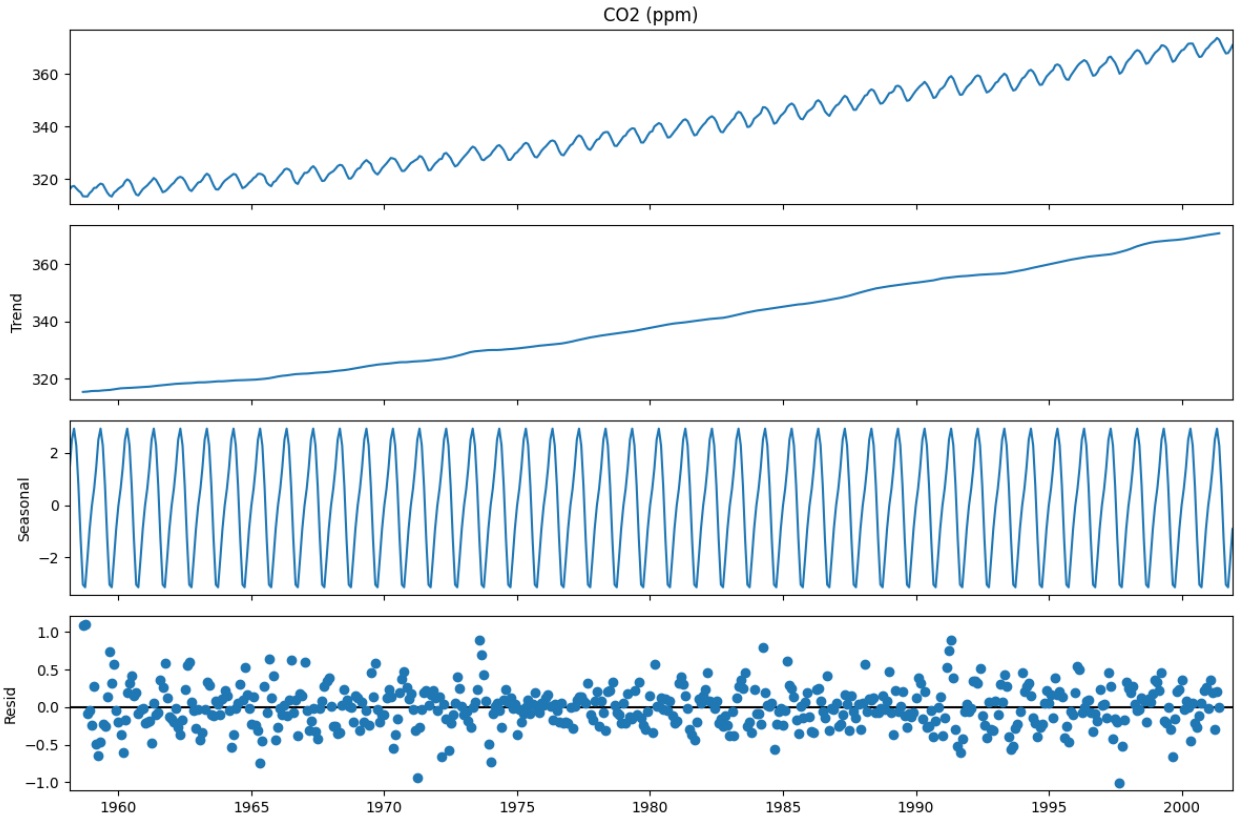
```
# Trend view with rolling statistics
plt.figure(figsize=(12, 4))
plt.plot(ts, label="Monthly CO2")
plt.plot(ts.rolling(12).mean(), label="12-month rolling mean",
color="red")
plt.plot(ts.rolling(12).std(), label="12-month rolling std",
color="green")
plt.title("CO2 Concentration with Rolling Statistics");
plt.ylabel("ppm"); plt.legend(); plt.show()

# Seasonal pattern: average level of each month after removing the
yearly trend
detr = ts - ts.rolling(12, center=True).mean()
detr.groupby(detr.index.month).mean().plot(kind="bar", figsize=(8, 3),
color="teal")
plt.title("Average Seasonal Effect by Month"); plt.xlabel("Month");
plt.ylabel("ppm"); plt.show()
```






```
# Decomposition into trend, seasonal and residual
dec = seasonal_decompose(ts, model="additive", period=12)
fig = dec.plot(); fig.set_size_inches(12, 8); plt.tight_layout();
plt.show()
```

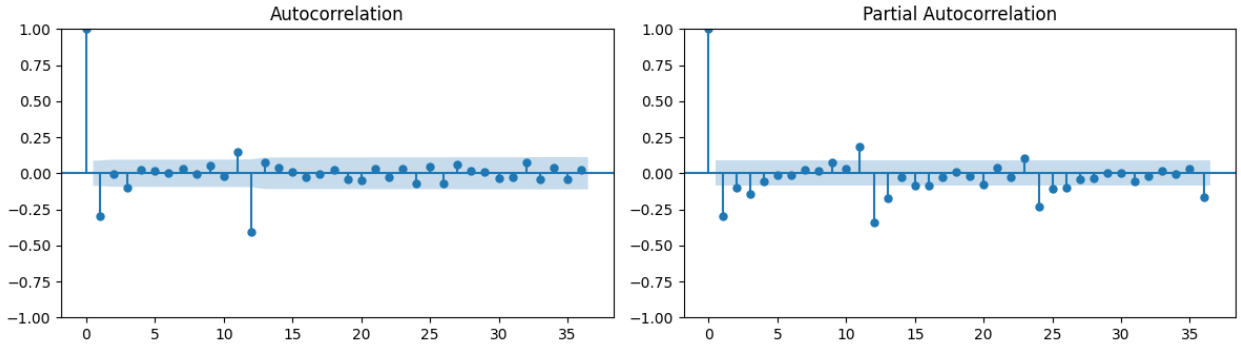


```python
# Stationarity check — ADF test
def adf(series, label):
    stat, p = adfuller(series.dropna())[:2]
    print(f"{label:<32} ADF = {stat:8.3f}   p-value = {p:.4f}   ->",
"Stationary" if p < 0.05 else "Not stationary")

adf(ts, "Original series")
adf(ts.diff(), "1st difference")
adf(ts.diff().diff(12), "1st + seasonal (12) difference")

stat_ts = ts.diff().diff(12).dropna()
fig, ax = plt.subplots(1, 2, figsize=(12, 3.5))
plot_acf(stat_ts, lags=36, ax=ax[0]); plot_pacf(stat_ts, lags=36,
ax=ax[1]); plt.tight_layout(); plt.show()
```

```
Original series                  ADF =    2.233   p-value = 0.9989   -
> Not stationary
1st difference                   ADF =   -4.752   p-value = 0.0001   -
> Stationary
1st + seasonal (12) difference   ADF =   -8.398   p-value = 0.0000   -
> Stationary
```




```python
# Train / test split — last 36 months for testing
h = 36
train, test = ts[:-h], ts[-h:]
print("Train:", train.index[0].date(), "to", train.index[-1].date(),
len(train))
print("Test :", test.index[0].date(), "to", test.index[-1].date(),
len(test))

preds = {}
# 1. Seasonal Naive baseline: value from the same month last year
preds["Seasonal Naive"] = pd.Series(train[-12:].values.tolist() * 3,
index=test.index)
# 2. Holt-Winters (additive trend + seasonality)
hw = ExponentialSmoothing(train, trend="add", seasonal="add",
seasonal_periods=12).fit()
preds["Holt-Winters"] = hw.forecast(h)
```

```
# 3. SARIMA
sar = SARIMAX(train, order=(1, 1, 1), seasonal_order=(1, 1, 1,
12)).fit(disp=False)
preds["SARIMA"] = sar.forecast(h)

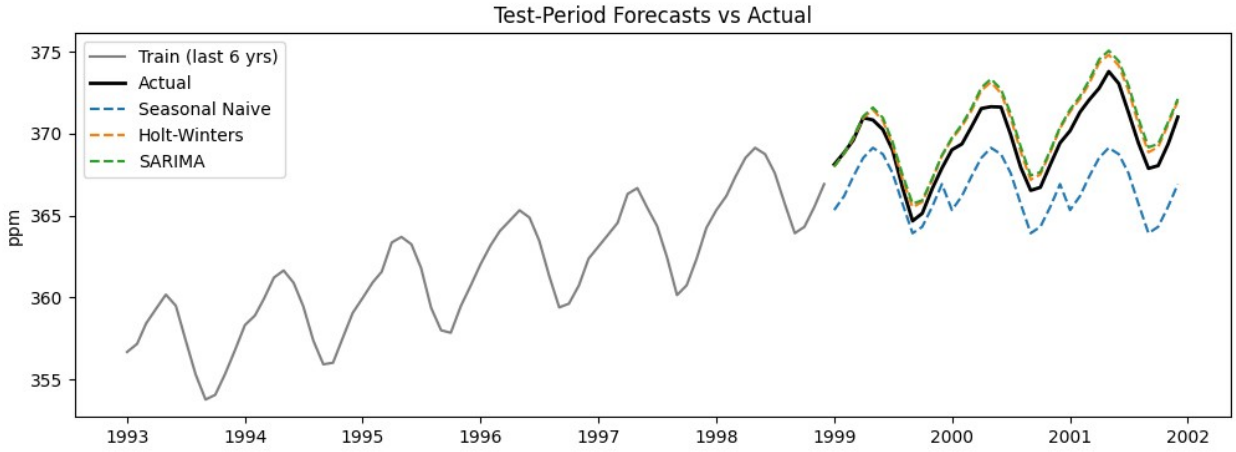
rows = []
for name, p in preds.items():
    rows.append([name, mean_absolute_error(test, p),
np.sqrt(mean_squared_error(test, p)),
                 np.mean(np.abs((test - p) / test)) * 100])
res = pd.DataFrame(rows, columns=["Model", "MAE", "RMSE", "MAPE
(%)"]).set_index("Model").round(4)
print(res)
best = res.RMSE.idxmin(); print("\nBest model (lowest RMSE):", best)
```

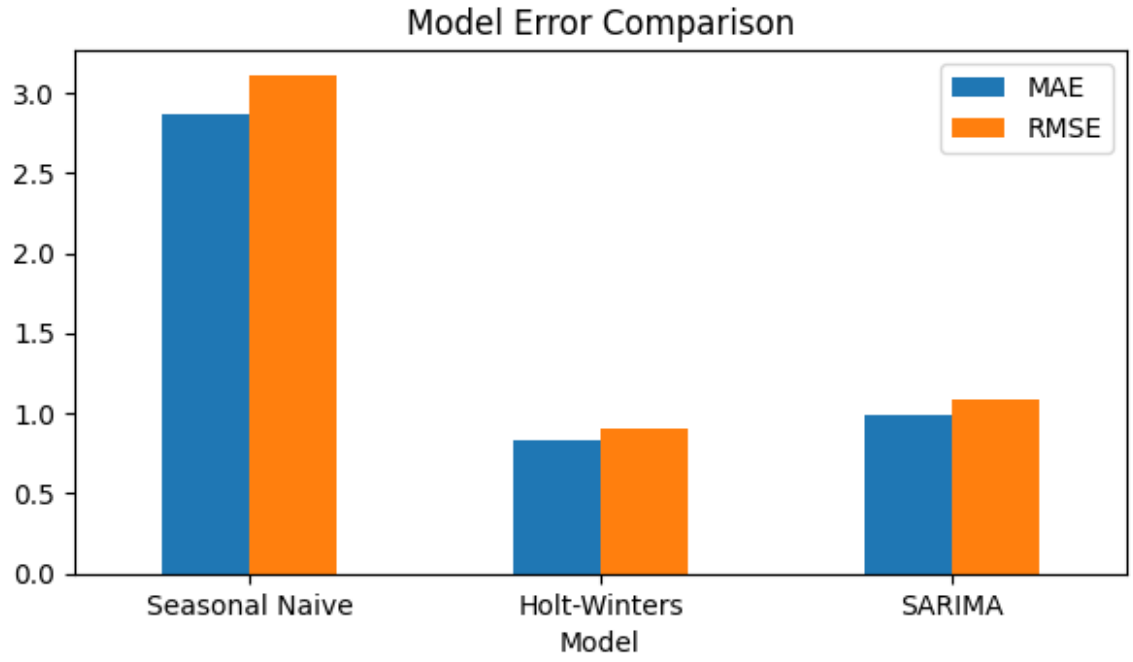
```
Train: 1958-03-01 to 1998-12-01 490
Test : 1999-01-01 to 2001-12-01 36
                   MAE    RMSE  MAPE (%)
Model
Seasonal Naive  2.8657  3.1088    0.7744
Holt-Winters    0.8264  0.9061    0.2235
SARIMA          0.9958  1.0845    0.2693

Best model (lowest RMSE): Holt-Winters
```

```
# Forecast vs actual on the test period
plt.figure(figsize=(12, 4))
plt.plot(train[-72:], label="Train (last 6 yrs)", color="gray")
plt.plot(test, label="Actual", color="black", linewidth=2)
for name, p in preds.items():
    plt.plot(p, "--", label=name)
plt.title("Test-Period Forecasts vs Actual"); plt.ylabel("ppm");
plt.legend(); plt.show()

res[["MAE", "RMSE"]].plot(kind="bar", figsize=(7, 3.5));
plt.title("Model Error Comparison"); plt.xticks(rotation=0);
plt.show()
```



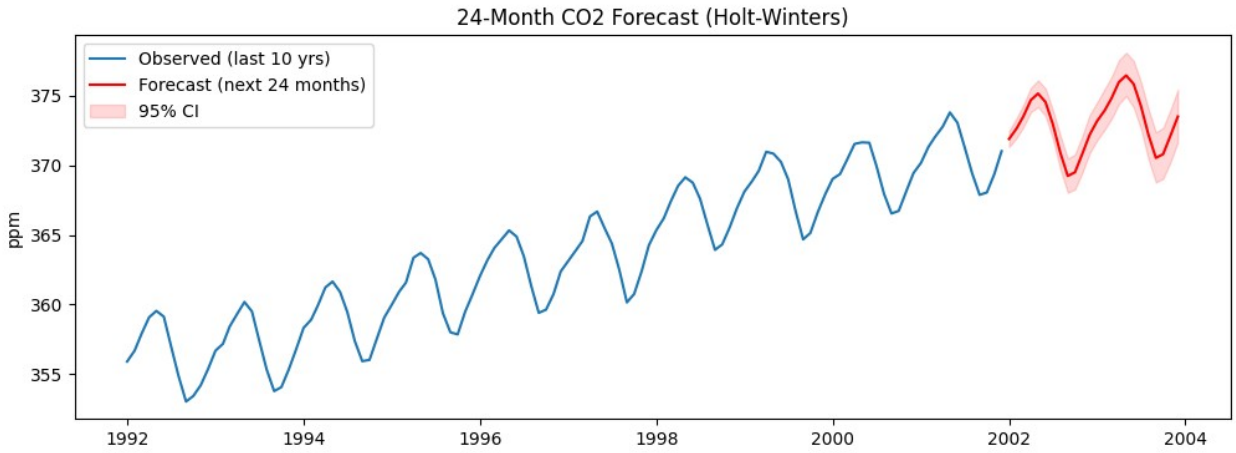



```
# Refit the best model on the full series and forecast the next 24
months with a 95% confidence interval
if best == "SARIMA":
    final = SARIMAX(ts, order=(1, 1, 1), seasonal_order=(1, 1, 1,
12)).fit(disp=False)
    fc = final.get_forecast(24); mean, ci = fc.predicted_mean,
fc.conf_int()
else:   # Holt-Winters (or baseline) -> Holt-Winters with simulated
intervals
    final = ExponentialSmoothing(ts, trend="add", seasonal="add",
seasonal_periods=12).fit()
    mean = final.forecast(24)
    sims = final.simulate(24, repetitions=1000, error="add",
random_state=42)
```

```
    ci = pd.DataFrame({"lower": sims.quantile(0.025, axis=1), "upper":
sims.quantile(0.975, axis=1)}, index=mean.index)

plt.figure(figsize=(12, 4))
plt.plot(ts[-120:], label="Observed (last 10 yrs)")
plt.plot(mean, label="Forecast (next 24 months)", color="red")
plt.fill_between(ci.index, ci.iloc[:, 0], ci.iloc[:, 1], color="red",
alpha=0.15, label="95% CI")
plt.title(f"24-Month CO2 Forecast ({best})"); plt.ylabel("ppm");
plt.legend(); plt.show()

out = pd.DataFrame({"Forecast": mean.round(2), "Lower 95%": ci.iloc[:,
0].round(2), "Upper 95%": ci.iloc[:, 1].round(2)})
print(out.head(12))
print(f"\nAverage yearly rise in the forecast: {(mean.iloc[-1] -
mean.iloc[-13]):.2f} ppm")
out.to_csv("CO2_Forecast_24_months.csv"); print("Saved
CO2_Forecast_24_months.csv")
```




```
            Forecast  Lower 95%  Upper 95%
2002-01-01    371.90     371.32     372.49
2002-02-01    372.64     371.97     373.40
2002-03-01    373.51     372.76     374.34
2002-04-01    374.69     373.83     375.57
2002-05-01    375.16     374.20     376.12
2002-06-01    374.55     373.54     375.52
2002-07-01    372.97     371.90     373.99
2002-08-01    370.92     369.74     372.15
2002-09-01    369.24     368.04     370.49
2002-10-01    369.50     368.28     370.78
2002-11-01    370.86     369.51     372.15
2002-12-01    372.21     370.88     373.64

Average yearly rise in the forecast: 1.30 ppm
Saved CO2_Forecast_24_months.csv
```

RESULT

Thus, the CO_2 time series is analysed for trend, seasonality and stationarity, and Seasonal Naive, Holt-Winters and SARIMA models are compared; the best model is used to forecast the next 24 months with a confidence interval.